

```

/// Node.java
package stLecture;
/*
 * Node class in Linked List
 * COP3330
 * Author: Safa
 */
public class Node {

    private int item;
    private Node next;

    // we dont need any default node in
    // our implementations
    // so we are stopping the developers calling
    // no arg constructor
    private Node(){
        next=null;
        item=0;
    }

    public Node(int gitem) {
        item=gitem;
        next=null;
    }

    public Node getNext() {
        return next;
    }

    public int getItem() {
        return item;
    }

    public void setNext(Node n) {
        next=n;
    }

    private void setItem(int gitem) {
        item=gitem;
    }
}

```

```
    }  
}
```

//Stack.java

```
package stLecture;  
// Stack interface implemented by Stack  
// implementation variations
```

```
public interface Stack {  
  
    public void push(int element);  
  
    public int peek();  
  
    public int pop();  
  
    public boolean isEmpty();  
  
}
```

//StackA.java

```
package stLecture;  
//Array implementation of Stack  
  
public class StackA implements Stack {  
    int top;  
    int[] arr;  
  
    public StackA() {  
        arr=new int[2];  
        top=0;  
    }  
  
    public void push(int y) {  
        if(!isOverflow()) {  
            arr[top]=y;  
            top++;  
        }  
    }  
}
```

```

    public int pop() {
        int returned=-1;
        if(!isEmpty()) {
            returned=arr[top-1];
            top--;
        }
        return returned;
    }

    public int peek() {
        int returned=-1;
        if(!isEmpty()) {
            returned=arr[top-1];
        }
        return returned;
    }

    public boolean isOverflow() {
        if(top>=arr.length) {
            System.out.println("Stack Overflow");
            return true;
        }
        return false;
    }

    public boolean isEmpty() {
        return top<=0;
    }

    public String toString() {
        StringBuilder sb=new StringBuilder("");
        for(int el:arr) {
            sb.append(el+" ");
        }
        return sb.toString();
    }
}
-----
//StackL.java

```

```

package stLecture;
//Linked List implementation of Stack

public class StackL implements Stack {

    Node top;
    StackL() {
        top=null;
    }

    public boolean isEmpty() {
        return top==null;
    }

    public String toString() {
        StringBuilder sb=new StringBuilder("");
        for(Node cur=getTop(); cur!=null;
cur=cur.getNext()) {
            sb.append(cur.getItem()+" ");
        }
        return sb.toString();
    }

    public void push(int y) {
        Node n=new Node(y);
        n.setNext(top);
        top=n;
    }

    public int pop() {
        int returned=-1;
        if(!isEmpty()) {
            returned=top.getItem();
            top=top.getNext();
        }else {
            System.out.println("Stack Empty");
        }
        return returned;
    }

    public int peek() {

```

```

        int returned=-1;
        if(!isEmpty()) {
            returned=top.getItem();
        }else {
            System.out.println("Stack Empty");
        }
        return returned;
    }
}

-----

//StackTester.java
package stLecture;

public class StackTester {

    public static void main(String[] args) {
        StackL sl=new StackL();
        StackA sa=new StackA();

        System.out.println("Testing LL version");
        testStack(sl);
        System.out.println("Testing array version");
        testStack(sa);
    }

    public static void testStack(Stack sl) {
        sl.push(3);
        sl.push(4);
        sl.push(5);

        System.out.println(sl.toString());
        int ret=0;
        for(int i=0;i<3;i++) {
            ret=sl.peek();
            System.out.println("PEEK "+ ret);
            ret=sl.pop();
        }
    }
}

```